

CS7DS2 Assignment - Week 4

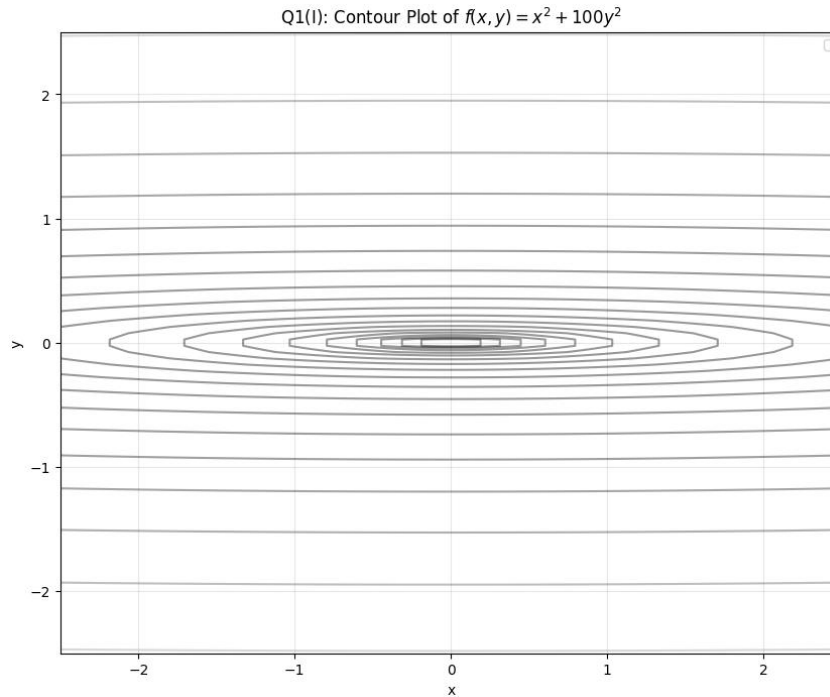
Name: Kehan Liu

ID:25334065

Q1

I

The objective function of this experiment is $f(x,y)=x^2+100y^2$ to make the original function easier to understand. To facilitate understanding of the function, I first plotted the original function to observe its shape and then analyzed it.



Observing the graph, we can see that the contour lines in the Y-direction of the function are very dense, while those in the X-direction are relatively sparse. This proves that when the function changes distance along the Y-axis, the change along the X-axis is relatively slow. Overall, the original function resembles a valley, with the y-axis representing the steep cliffs on both sides and the x-axis representing the valley floor with only minor undulations.

The problem requires performing gradient descent using four given optimization algorithms. Therefore, the first step is to calculate the gradient of the original function: taking the partial derivative $\frac{\partial f}{\partial x}=2x$ with respect to x and $y \frac{\partial f}{\partial y}=200x$, we obtain the gradient of the function

$\nabla f=[2x,200y]^T$. This means that the gradient is 100 times steeper in the y-direction than in the x-direction. Then, the four optimization algorithms are applied.

Polyak Step Size : The essence of this optimization algorithm is to calculate the optimal step size in each iteration based on the distance between the current function value and the optimal value. Its mathematical principle is that when the function is at a certain point, the function height is known, and we also know that the original function has a minimum point (0,0). According to the

mathematical formula $g = \frac{h}{d}$ (g is the slope, d is the horizontal distance, and h is the vertical height),

our goal is to find the step size, which is d , by simply transforming the above formula $d = \frac{h}{g}$.

In the gradient descent algorithm, Movement vector=Step Size×Gradient we want the length of this movement vector to be exactly equal to the ideal distance d mentioned above. Therefore , we obtain

Step Size× $g = \frac{h}{g}$ the final result as shown below Step Size= $\frac{h}{g \times g} = \frac{h}{g^2}$. This is the core code of the function below. After calculating the ideal step size, the position is updated, and the iteration continues.

```
def run_polyak ( w_init , iterations ):  
    w = w_init.copy ()  
    loss_history = []  
    for t in range ( iterations ):  
        loss = func ( w )  
        loss_history.append ( loss )  
        grad = gradient ( w )  
        grad_norm_sq = np.sum ( grad ** 2 )  
        if grad_norm_sq == 0 :  
            break  
        step_size = loss / grad_norm_sq  
        w = w - step_size * grad  
    return loss_history
```

RMSProp : Because the steepness of the original function differs greatly in the x and y directions, a single uniform step size is insufficient to simultaneously satisfy the movement in both directions. Therefore, we need a parameter to adjust the step size in both directions. The core mechanism of the algorithm is to dynamically adjust the step size in each dimension using the exponential moving average of the squared gradient. Specifically, the algorithm updates a variable s by $s_t = \beta s_{t-1} + (1-\beta)g_t^2$ accumulating historical gradient information and incorporating new gradient information . When updating parameters, the current gradient is divided by $\sqrt{s}$, thus achieving adaptive adjustment.

In the context of the objective function of this problem, the update function is $w_{\text{new}} = w_{\text{old}} - \frac{\alpha}{\sqrt{s}} \cdot \text{grad}$:

For the y -axis, grad is large , and the denominator $\sqrt{s}$ is also large , which suppresses oscillations and makes position updates in this direction more stable.

For the X -axis, grad is very small , and the denominator $\sqrt{s}$ is also small , which amplifies the stride and makes it faster on flat terrain.

```
def run_rmsprop ( w_init , iterations , alpha , beta , epsilon = 1e-8 ):  
    w = w_init.copy ()  
    s = np.zeros_like ( w )  
    loss_history = []  
    for t in range ( iterations ):  
        loss_history . append ( func ( w ))  
        grad = gradient ( w )  
        s = beta * s + ( 1 - beta ) * ( grad ** 2 )  
        w = w - alpha * grad / ( np.sqrt ( s ) + epsilon )
```

```
return loss_history
```

Heavy Ball : As its name suggests, this algorithm is like placing a heavy ball and letting it roll in a valley. It has inertia; after reaching the lowest point, it won't stop immediately but will move a distance in its original direction. In its implementation, the algorithm introduces a velocity vector v .

The update rule is $v_{t+1} = \beta v_t - \alpha \nabla f(w_t)$ to retain a portion of the original velocity vector, add a certain amount of the current gradient, and accelerate or decelerate the ball. The position is then updated based on this velocity vector v ($w = w + v$).

In the objective function of this problem, the gradient oscillates rapidly between positive and negative values along the steep y -direction. The velocity vector v adds these time-varying vectors in opposite directions, causing them to cancel each other out. This effectively smooths the trajectory.

In the flat x -direction, the gradient always points to the minimum. The momentum term accumulates these directionally consistent updates, causing the velocity vector v to increase. This allows the algorithm to move faster in valleys.

```
def run_heavy_ball ( w_init , iterations , alpha , beta ):  
    w = w_init.copy ()  
    v = np.zeros_like ( w )  
    loss_history = []  
    for t in range ( iterations ):  
        loss_history . append ( func ( w ))  
        grad = gradient ( w )  
        v = beta * v - alpha * grad  
        w = w + v  
    return loss_history
```

Adam : This algorithm is more like a combination of Heavy Ball and RMSProp. Each update step of Adam contains four key steps:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

momentum in a heavy ball. When the gradient direction is continuous and consistent, m_t accumulates and increases, thus accelerating convergence; when the gradient oscillates randomly, the positive and negative values cancel m_t each other out, causing it to decrease and thus suppressing oscillations.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

v_t It is a weighted average of the squared gradients over time, reflecting the momentum of the gradient in that dimension, and is used for subsequent denominator normalization. If the gradient in a certain dimension is consistently large, v_t the step size will be reduced; conversely, it will be increased.

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Since the first and second moment estimates (m and v) are initialized to 0, they are biased towards zero in the early stages of iteration. Therefore, dividing them by $(1 - \beta^t)$ amplifies these estimates to correct this initialization bias. For example, when $t=1$ and $\beta_1=0.9$, dividing the first moment estimate m_1 by 0.1 is equivalent to amplifying it by a factor of 10. This crucial step prevents the parameter update step size from being excessively small during the initial state, effectively avoiding a slow

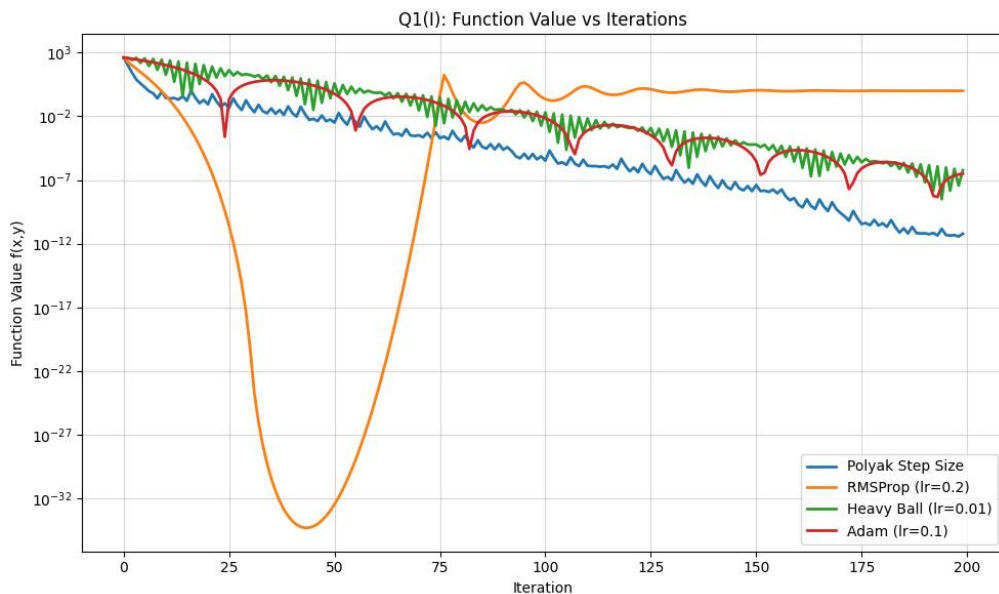
"cold start." As the iteration count t increases, the denominator $(1-\beta^t)$ approaches 1, and the bias correction term naturally decays and becomes ineffective.

$$w_{t+1} = w_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Finally, update the location.

```
def run_adam ( w_init , iterations , alpha , beta1 , beta2 , epsilon = 1e-8 ):
    w = w_init.copy ()
    m = np.zeros_like ( w )
    v = np.zeros_like ( w )
    loss_history = []
    for t in range ( 1 , iterations ) + 1 ):
        loss_history . append ( func ( w ))
        grad = gradient ( w )
        # Adam update
        m = beta1 * m + ( 1 - beta1 ) * grad
        v = beta2 * v + ( 1 - beta2 ) * ( grad ** 2 )
        # Bias correction
        m_hat = m / ( 1 - beta1 ** t )
        v_hat = v / ( 1 - beta2 ** t )
        w = w - alpha * m_hat / ( np.sqrt ( v_hat ) + epsilon )
    return loss_history
```

Applying the four algorithms above, gradient descent is performed on the objective function, iterating 200 times, and the results are visualized as follows.



Polyak Step Size (blue line): The Polyak algorithm exhibits the most stable and consistent convergence speed. In the graph, it appears as a slightly fluctuating straight line that steadily decreases. By the 200th iteration, the function value has dropped to approximately 10^{-12} . This is because we know the global optimum $(0, 0)$ in this problem, and the algorithm utilizes this information. It's worth noting that it exhibits frequent, small oscillations rather than a perfectly smooth descent. I believe this is because while the Polyak formula calculates the optimal step size along the current gradient direction, the gradient direction itself is local. In a narrow valley, the gradient primarily points towards the steep valley walls, not towards the global minimum. Therefore ,

each step of the algorithm is perpendicular to the direction of the previous step. This directional correction results in the fluctuations observed in the function value graph .

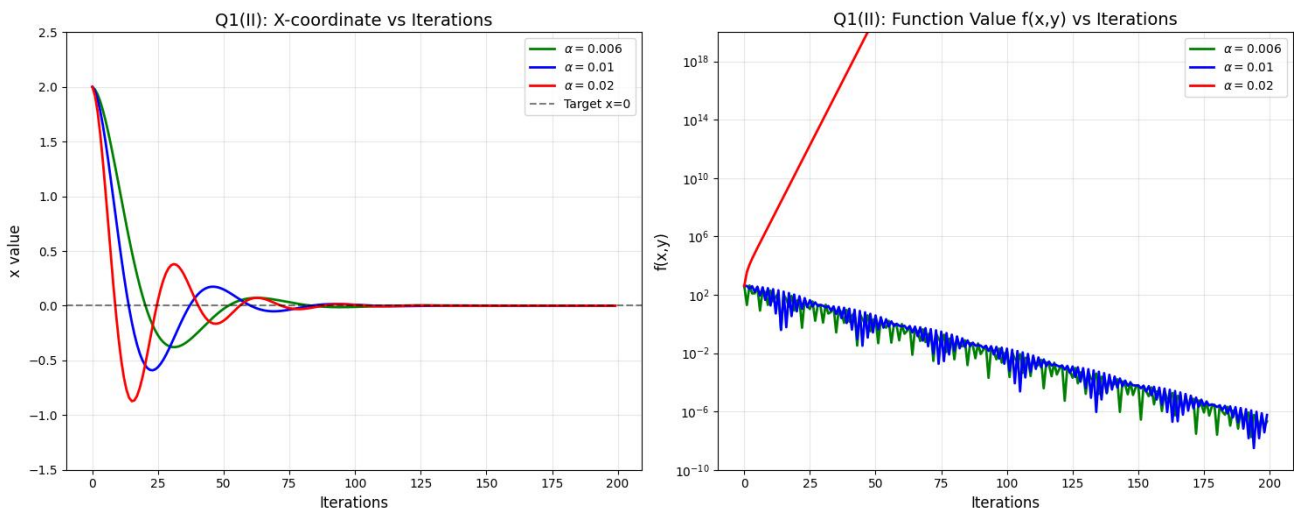
RMSProp (orange line): The RMSProp descent algorithm can be broken down into two phases. Descent phase (0-45 steps): It converges rapidly, reaching the limit of machine accuracy around step 45, much faster than all other algorithms. Bounce phase (45-80 steps): After hitting the bottom, it bounces and stabilizes within 10^{-1} to 10^0 the range, exhibiting highly unstable performance. I believe this is because RMSProp normalizes the gradient by dividing by the moving average of squared gradients . Combined with a large learning rate $\sqrt{s}\alpha=0.2$, it can quickly approach the endpoint in a flat X-axis direction. As for the bounce, I think it's due to the fixed step size. When the algorithm reaches near the bottom, the gradient approaches 0, and S in the denominator becomes extremely small. At this point, the step size is too large for such a small region. The algorithm cannot stop at the bottom.

Heavy Ball (green line) & Adam (red line): These two curves exhibit clear oscillations. Heavy Ball shows oscillations with a generally slow downward trend. Adam oscillates at a lower frequency. The overall convergence trend is similar to Heavy Ball, eventually settling around 10^{-6} . I believe this regular oscillation phenomenon is due to Heavy Ball and Adam using momentum to accelerate their movement along the gentle X- axis, but this inertia causes overshooting along the steep Y-axis . This causes the trajectory to swing back and forth between the valley walls, thus producing the oscillating waveform shown in the figure.

In summary, the Polyak step size offers the best balance between stability and speed. RMSProp converges the fastest, but also demonstrates that large step sizes can lead to bouncing of the final solution in the absence of a step size decay strategy. Heavy Ball and Adam successfully utilize momentum to accelerate movement in the X direction, but exhibit severe oscillations in the Y direction, resulting in continuous fluctuations on the convergence curve.

II

The code is also reused from Q1.1. Based on the problem requirements, we will use $\alpha = 0.006$, 0.01 , and 0.02 to observe the different effects on the gradient descent of the objective function and visualize them. See the figure below. I have drawn two graphs: one showing the change of X with iteration, and another showing the change of $F(X,y)$ with iteration to observe the instability.



Left image: Changes in the X-axis coordinate

As can be seen, the three lines show only slight differences in amplitude. The amplitude increases with increasing α , but they all exhibit the same trend: after a brief oscillation, they approach 0 after approximately 80 iterations. This is because the objective function in the X direction is very flat; for X, even with $\alpha=0.2$, it's a very safe small step size. However, this number is completely inapplicable for the y direction. This can be verified in the right-hand figure.

Right figure: Changes in function values

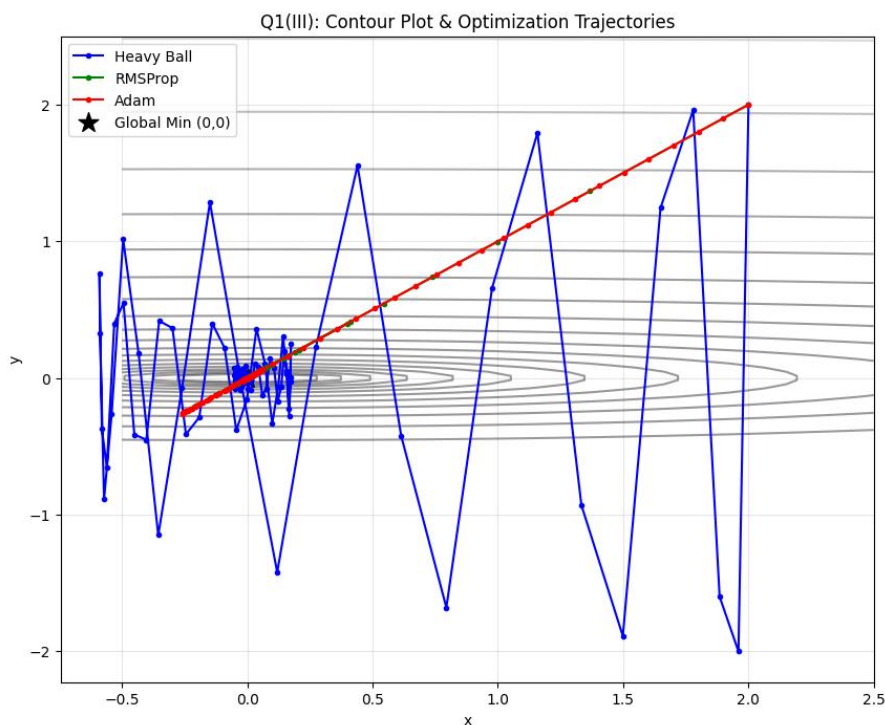
The green and blue lines both exhibit a downward oscillating trend, with very similar trends. Overall, the two lines overlap significantly. They eventually iterate to 10^{-6} . As for the oscillation, as analyzed in Q1.1, inertia in the steep y-axis direction leads to overshooting. This causes the trajectory to oscillate back and forth between the two sides of the valley, thus producing the oscillating waveform shown in the figure.

Focus on the red line: after just a few iterations, the red line explodes, with the function value skyrocketing to above 10^{18} . In the left graph, we already know that in the x-direction, $\alpha=0.2$ is a very safe small step size and doesn't cause a sudden change in the X value. Therefore, the problem must lie in the y-direction. Because the y-direction is too steep, $\alpha=0.2$ is too large. Due to the excessively large step size, the algorithm cannot slow down when bouncing back and forth between the steep valleys of the y-axis. Each iteration not only fails to fall to the bottom but is instead bounced higher than the previous step by the huge backward gradient. This error is amplified exponentially in the y-direction, causing the overall function value to collapse.

This shows that for such extreme functions, the Heavy Ball algorithm is very sensitive to the learning rate. A slightly large learning rate may directly cause the algorithm to crash, but a learning rate that is too small will affect the algorithm speed.

III

Draw contour lines for the objective function and overlay the trajectories of Heavy Ball $\beta=0.9, \alpha=0.01$, RMSProp $\beta=0.9, \alpha=0.2$, and Adam $\beta_1=0.9, \beta_2=0.999, \alpha=0.1$. The specific algorithm implementation is the same as the previous two problems. In this experiment, I iterated the above three algorithms 100 times, and the image is as follows.



The trajectory of the Heavy Ball (blue line) exhibits a sawtooth-like oscillation. Starting from (2,2), it initially rushes almost vertically to $y=-2$. Subsequently, it bounces violently back and forth between the two sides of the valley, each bounce almost returning to the highest point on the opposite side, while its movement along the X-axis is very slow. This is because the Heavy Ball's learning rate is consistent in both the x and y directions. In the y-direction, the gradient is too large, generating enormous velocity. Inertia prevents the algorithm from stopping at the bottom of the valley, causing it to oscillate continuously.

The RMSProp (green line) and Adam (red line) trajectories are almost identical, forming a near-perfect straight line that travels directly from the starting point (2,2) to the ending point (0,0). It exhibits no oscillations. Notably, upon reaching (0,0), neither line stops immediately but continues moving in its original direction for a short distance before returning to the lowest point. This is due to their adaptive scaling properties. For RMSProp , in the y-direction, the gradient is large, and the exponential moving average of the squared gradient is also large, resulting in a smaller step size to prevent oscillations . In the x-direction, the gradient is small , and the exponential moving average of the squared gradient is also small, accelerating the movement . Adam, in addition to using the mean of the squared gradient for adaptive scaling like RMSProp , also utilizes the mean of the gradient. This adds inertia to the algorithm, preventing Adam from stopping immediately upon reaching the lowest point (0,0) . Instead, it continues moving in its original direction for a short distance due to inertia before returning to the lowest point .

In summary, this demonstrates that a fixed learning rate is ineffective for handling such special functions. RMSProp and Adam, on the other hand, exhibit strong adaptability . By independently rescaling the gradient in each dimension, they transform a complex ill-conditioned optimization problem into a simple linear descent problem. Trajectory analysis ultimately shows that for problems with extremely large curvature differences, adaptive step size is not only a performance optimization but also a necessary condition for achieving efficient and stable convergence.

Q&A

why the function causes zig-zag behavior

the objective function $f(x,y)=x^2+100y^2$ are an extremely narrow ellipse. The valleys are very steep along the y-axis and very gentle along the x-axis, as discussed in Q1.1.

Gradient descent always moves in the opposite direction of the current gradient. On the sidewall of a narrow valley, the gradient vector points almost entirely to the opposite side of the valley, with a very small deviation towards the lowest point at the bottom.

This causes the algorithm to lag behind in each iteration. Since the learning rate is globally uniform, the step size cannot be too small to move in the X direction; however, this is often too large for the Y direction, causing the algorithm to jump over the valley floor in one step and end up on the opposite mountain wall. This phenomenon of moving back and forth between two steep mountain walls forms the Zig-Zag trajectory.

how momentum affects oscillations

Given that the Zig-Zag oscillations move back and forth between two steep cliffs, the sign of the gradient in the y-direction flips at each step.

The momentum term $v_{t+1}=\beta v_t+(1-\beta)g_t$ is a weighted average of historical gradients. When the gradient sign frequently flips, positive and negative gradients cancel each other out during

accumulation. Therefore, momentum helps to counteract oscillations and make the trajectory smoother.

However, if the step size is too large α , such as in Q1 $\alpha=0.02$, momentum introduces inertia. Even after reaching the bottom, the enormous kinetic energy will propel it up the opposite slope, perhaps even higher than before. This is the phenomenon of the surge in function value observed in Q1.2. This exacerbates the system's instability, leading to increasingly larger oscillations.

how adaptive methods respond to different curvature in x and y ,

The adaptive method calculates the step size separately for each parameter. They are then divided by the root mean square of the historical gradients $\sqrt{v_t}$.

Analyzing this problem within the context of the problem itself, the gradient on the Y -axis is very large, and the cumulative squared term is v_y also very large, $\frac{\alpha}{\sqrt{v_y}}$ significantly reducing the effective step size. This is equivalent to reducing the step size in steep directions to prevent Zig-Zag oscillations.

On the X -axis, the gradient is very small, the cumulative squared term v_x is small, and the effective step size $\frac{\alpha}{\sqrt{v_x}}$ is relatively amplified. This is equivalent to accelerating convergence in the flat direction.

Through this anisotropic scaling, the adaptive algorithm effectively normalizes the narrow elliptical valley. In the algorithm's view, the steepness in the x and y directions becomes consistent, allowing it to reach the lowest point faster and more steadily.

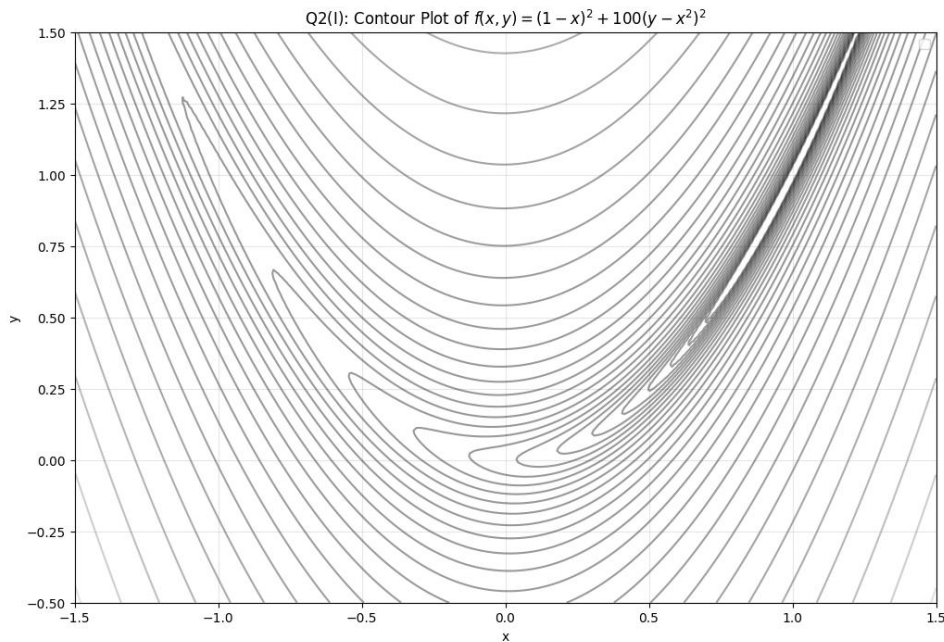
why ill-conditioning influences the stability limit for α .

In this problem, the gradient is extremely large along the Y -axis and extremely small along the X -axis. For algorithms using a globally uniform learning rate, α the upper bound of the learning rate is forced to be very low to prevent divergence when navigating between steep sections in the Y -axis. If the learning rate is increased to accelerate in the gentler X -axis α , this step size becomes extremely large in the Y -axis, not only failing to reach the bottom but instead bouncing directly to a higher position on the opposite side. This is the phenomenon we observed in Q1.2, where the function value diverged because the stability limit along the Y -axis was exceeded. Therefore, the gradient difference in the ill-conditioned terrain forces the algorithm $\alpha=0.02$ to be extremely sensitive to the steepest direction, limiting the upper bound of the learning rate adjustment.

Q2

I

The objective function of this experiment is $f(x,y)=(1-x)^2+100(y-x^2)^2$: To facilitate understanding of the function, I first plotted an isograph of the original function to observe its shape and then analyzed it.

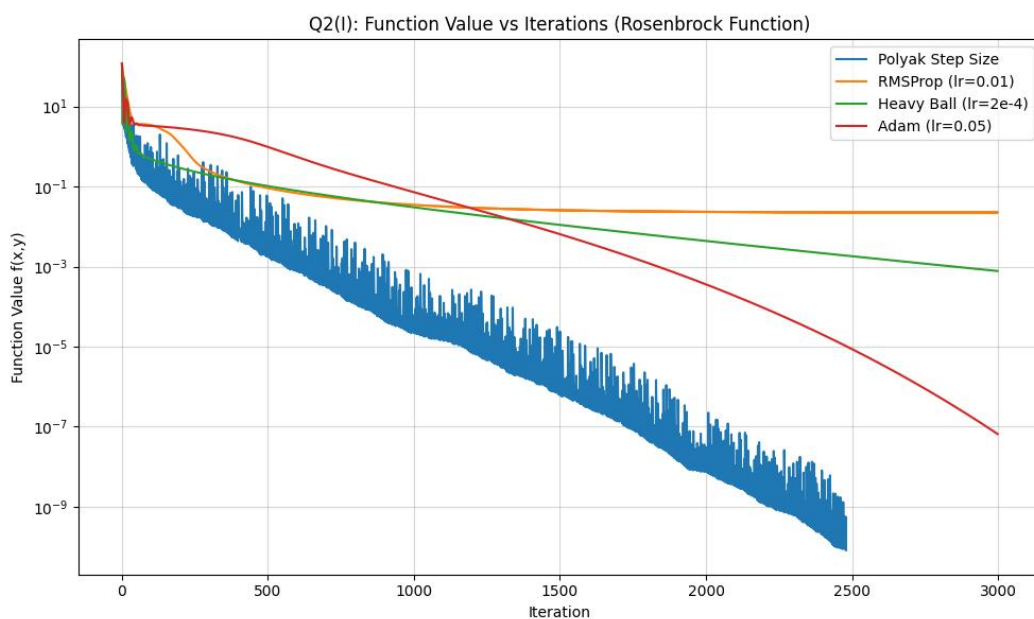


Observing the image, we can see that the overall outline of the function presents a curved, crescent-shaped structure. Similar to Q1, the contour lines along the Y-axis are very dense, indicating a maximum gradient and a very steep slope in that direction. However, unlike the straight valley in Q1, the valley floor in this problem is a curved parabola. Along the direction of this parabolic valley floor, the contour lines are extremely sparse, and the terrain is extremely flat. The global minimum is located at (1, 1).

The problem requires performing gradient descent using four given optimization algorithms. First, the gradient of the original function needs to be obtained: take the partial derivative

$$\frac{\partial f}{\partial x} = -2(1-x) - 400x(y-x^2) \text{ with respect to } x \text{ and } y \quad \frac{\partial f}{\partial y} = 200(y-x^2) \text{ to get the gradient of the function.}$$

Then, the four optimization algorithms are applied. The implementation of the optimization algorithms is the same as in Q1, with a Polyak step size ($\alpha = 0.1$), and the other parameters are configured as required by the problem. The results are shown in the figure below.



Polyak (blue): Converges fastest, eventually reducing the error to 10^{-9} the order of magnitude. However, visually, this line is not a thin line, but a thick band of color. This indicates that the

function value oscillates violently between each iteration. I believe this is because the Polyak step size is calculated based on the theoretically optimal step size using the current gradient and function value. However, the valley of the original function is curved; moving along the current gradient direction often results in directly hitting the wall opposite the valley, rather than proceeding along the curve. Therefore, the algorithm oscillates violently, bouncing back and forth on both sides of the valley. However, because the algorithm is calculated based on the known minimum point, it is still the fastest.

RMSProp (orange) : This algorithm performed the worst in this experiment. In the first 1000 iterations, the algorithm slowly decreased to the order of 10^{-1} magnitude, but then it stopped there, remaining almost unchanged for the next 2000 steps, resembling a straight line. The final result was the worst among all algorithms. I believe this is because after the algorithm quickly reached the bottom in the early stages, the bottom was too flat, and the gradient was very small. The exponential moving average of the squared gradient of RMSProp decreased, leading to a larger update step size. When the algorithm approached the optimal solution, the step size was too large, making it unable to accurately reach the lowest point, causing it to stagnate around the lowest point.

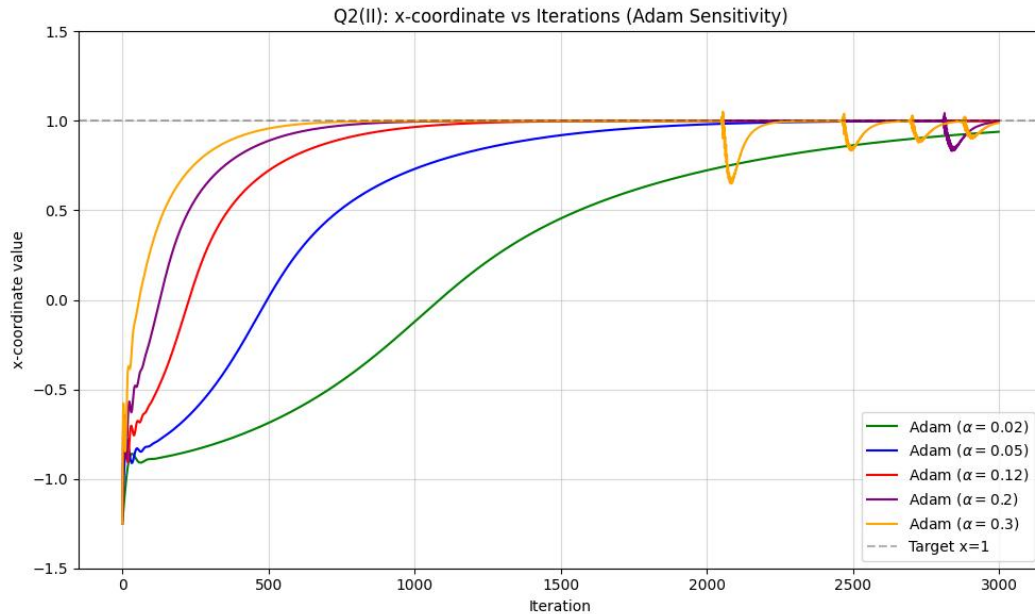
Heavy Ball (green) : The Heavy Ball appears as a very smooth, monotonically descending straight line. It has no oscillations, but the downward slope is relatively gentle, eventually converging to 10^{-3} . I believe this is because the learning rate is too low, limited by a very small step size, resulting in very slow movement.

Adam (red) : Adam starts slower than RMSProp, but it maintains a consistent and stable downward trend. Around step 1300, it surpasses RMSProp and eventually reaches 10^{-7} , and approaches the accuracy of Polyak, without the dramatic oscillations of Polyak. This is thanks to its adaptive scaling mechanism.

This experiment shows that, under the current objective function, Polyak remains the fastest and best algorithm, but at the cost of extreme instability. Adam further demonstrates that it is the algorithm best suited for complex terrain. The other two algorithms perform only moderately.

II

The implementation of the Adam algorithm has been explained in detail in Q1.1. This question only explores the impact of the learning rate on the Adam algorithm. For the Adam algorithm, with $\beta_1=0.9$ and $\beta_2=0.999$ fixed, and learning rates selected: $\alpha=0.02, 0.05, 0.12, 0.2, 0.3$, the curve of x-coordinate vs. number of iterations is shown in the figure below.



The green line ($\alpha=0.02$) shows an extremely gentle upward trend. Although it is very stable without any oscillations, at the end of 3000 iterations, it only reached $x \approx 0.95$, and failed to fully converge to the target value $x=1$.

The blue line ($\alpha=0.05$) shows a trend almost identical to the green line, but it converges much faster, successfully reaching the target value $x=1$ around step 2000.

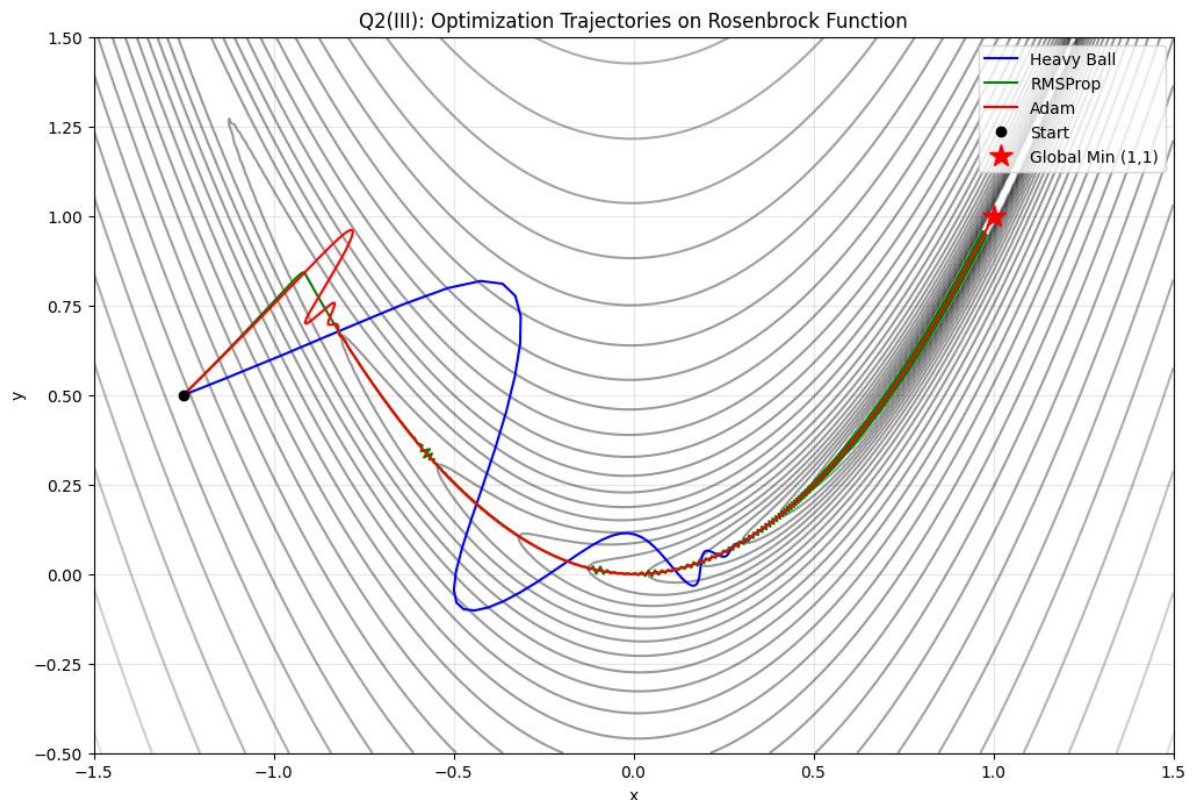
The red line ($\alpha=0.12$) It converges fastest in the early stages, reaching $x=1$ around step 1000. Notably, in the first 50 steps, due to the large learning rate, the red line exhibits noticeable wave-like oscillations. However, once it enters a flat valley, the speed gained from the large step size allows it to quickly outpace other algorithms.

Purple line ($\alpha=0.2$) and yellow line ($\alpha=0.3$) These two extreme learning rates reveal the stability limits of Adam. Although they approach $x=1$ very quickly, oscillations occur later. This indicates that when approaching the narrow trough near the minimum, an excessively large learning rate causes the algorithm to repeatedly jump out of the trough, failing to pinpoint the exact location.

Unlike the Heavy Ball algorithm discussed in Q1.2, Adam exhibits strong robustness when faced with large learning rates. Its adaptive denominator effectively prevents early gradient explosion in steep regions, accelerating the approach to the minimum point. This directly demonstrates Adam's greater adaptability to complex terrains. However, the later oscillations of the purple and yellow lines also indicate that an excessively large learning rate can still compromise the final convergence accuracy.

III

The iteration paths of Heavy Ball, RMSProp, and Adam are overlaid on the contour plot of the Rosenbrock function. The implementation logic is the same as before and will not be described in detail. The result is shown in the figure below.



Heavy Ball (blue) : In the early stages , it did not quickly enter the valley. Instead, it rushed out of the bend, drew a huge arc, and oscillated several times before falling into the valley. It then moved along the valley floor and ultimately did not reach the finish line.

RMSProp (Green) : The RMSProp track largely follows Adam 's pattern , but fails to reach the endpoint. Initially, it cuts directly into the valley from the starting point without oscillation . However, once it enters the flat valley , its movement becomes extremely slow, ultimately failing to reach the endpoint because the gradient is minimal at the bottom. This results in an exponential moving average of the squared gradient , amplifying the update step size . This causes the algorithm to bounce violently back and forth between the narrow valley walls , represented by the thick green line at the end, ultimately failing to reach the endpoint.

Adam (Red) : Adam's track is the best of the three. It exhibits some fluctuations in the early stages . Unlike RMSProp, Adam does not stall after entering the trough ; it moves smoothly along the crescent-shaped ridge and eventually reaches the lowest point.

Experimental results demonstrate that, when dealing with complex terrain, single-mechanism algorithms often struggle to balance convergence speed and stability. The Adam algorithm, by dynamically adjusting the step size and utilizing momentum to smooth the trajectory, exhibits the strongest performance and is the best choice for handling such problems .

Q&A

why Rosenbrock has slow convergence for basic methods (narrow curved valley)

As we analyzed in Q2.1, the Rosenbrock function has the shape of a very narrow, steep canyon with a flat bottom that curves in a parabolic shape.

Basic gradient descent relies entirely on the current negative gradient direction. At any point within a valley, the gradient vector is almost perpendicular to the valley wall, while the component pointing towards the global minimum along the valley floor is extremely small. To move along the flat valley floor, the algorithm requires a large step size. However, if the step size is too large, the algorithm will immediately crash into the steep valley wall and bounce back to the other side — the Zig-Zag phenomenon. To maintain stability, a very small step size must be used. This results in the algorithm only being able to advance slowly along the valley floor .

how momentum changes oscillations/overshoot

Oscillations: On the cross-section of the valley, the gradient direction constantly switches between positive and negative. The momentum term, by accumulating historical gradients, causes the gradients in the positive and negative directions to cancel each other out. This suppresses invalid oscillations perpendicular to the valley floor, helping the algorithm stabilize near the valley floor centerline.

Overshoot: Because the valley floor of the original function is curved, momentum attempts to maintain the algorithm's linear motion . When the valley curves, the algorithm cannot turn in time due to inertia, thus crashing out of the valley floor. This corresponds to the reason why the Heavy Ball trajectory draws a huge arc in Q2(III). Although momentum increases the speed, it also introduces overshoot .

how adaptive scaling affects progress along the valley

The adaptive method (RMSProp, Adam) $\sqrt{v_t}$ adjusts the step size for each dimension by dividing by the root mean square of the gradient.

In the y-direction : a large gradient results in a large denominator , and the step size is compressed. This prevents the algorithm from hitting the valley walls.

In the x-direction : the gradient is small , the denominator is small , and the step size is amplified. This accelerates the algorithm's progress towards the endpoint.

This mechanism effectively transforms the long, winding, banana-shaped valley into a more uniform circular bowl. The algorithm no longer perceives any particular direction as exceptionally steep or flat, thus enabling it to move along the valley floor at a more consistent speed and avoiding the stagnation issues present in the basic method.

why the stability limit for α is sensitive to curvature

Curvature essentially reflects the steepness of a function . In Rosenbrock, the direction perpendicular to the valley walls is extremely steep, while the direction along the valley floor is extremely gentle.

The greater the curvature, the smaller the maximum allowable step size must be. ($\alpha < 2/\lambda_{\max}$) .

In the terrain of this problem , if the step size is slightly increased in flat areas to speed things up α , it will cause the huge gradient multiplied by this step size to result in a single update covering a physical distance exceeding the entire canyon wall. The algorithm will not only fail to reach the valley floor but will also be launched directly onto the opposite high wall. Therefore, α the step size is extremely sensitive to high curvature and must be set to a very small value.